Chapitre 7 — Adresses mémoire, pointeurs et références

Prérequis : chapitres 1 à 6 · **Exercices :** `E07_Adresses_pointeurs_references`

Objectifs

- obtenir l'adresse d'une variable, déclarer et déréférencer un pointeur ;
- utiliser un pointeur en paramètre et vers une structure (`->`) ;
- comparer valeur, référence et pointeur, notamment pour transmettre un paramètre.

Sommaire

- Chapitre 7 — Adresses mémoire, pointeurs et références
  - Objectifs
  - Sommaire
  - 1. Adresses et pointeurs
    - 1.1 Le pointeur nul
  - 2. Pointeurs et fonctions
  - 3. Pointeurs et tableaux
  - 4. Pointeur vers une structure : `->`
  - 5. Valeur, référence ou pointeur ?
    - 5.1 Comparaison des notions
    - 5.2 Trois façons de transmettre un paramètre
  - Erreurs fréquentes
  - Synthèse

1. Adresses et pointeurs

Chaque variable occupe un emplacement mémoire repéré par une **adresse**. `&x` donne l'adresse de `x`. Un **pointeur** est une variable qui contient une adresse : `int*` se lit « pointeur vers un `int` ». L'opérateur `*` appliqué à un pointeur donne accès à la variable pointée : c'est le **déréférencement**.

```cpp
#include <iostream>

int main() {
    int temperature{25};
    int* ptr{&temperature};      // ptr contient l'adresse de temperature

    std::cout << *ptr << '\n';   // lecture de la variable pointee : 25

    *ptr = 30;                   // modification de la variable pointee
    std::cout << temperature << '\n';

    return 0;
}
```

Résultat :

```
25
30
```

```
 temperature           ptr
+-----------+      +------------------------+
|    30     | <--- | adresse de temperature |
+-----------+      +------------------------+
```

Les symboles `&` et `*` changent de sens selon l'endroit où ils sont écrits :

Écriture	Sens
`&temperature` (devant une variable)	Adresse de la variable
`int& ref` (dans une déclaration)	Référence (chapitre 5)
`a & b` (entre deux entiers)	ET bit à bit (chapitre 6)
`int* ptr` (dans une déclaration)	Type pointeur
`*ptr` (dans une expression)	Variable pointée

1.1 Le pointeur nul

Un pointeur qui ne désigne rien vaut `nullptr`. On le teste **avant** de le déréférencer.

```cpp
int* capteur{nullptr};               // ne designe encore rien

if (capteur != nullptr) {
    std::cout << *capteur << '\n';   // execute seulement si capteur designe une variable
}
```

> **Interdit :** déréférencer un pointeur nul ou non initialisé est un **comportement indéfini**, souvent un plantage. On initialise donc toujours un pointeur, au besoin avec `nullptr`.

2. Pointeurs et fonctions

Un pointeur peut être transmis à une fonction. Si le pointeur désigne une variable, la fonction peut la modifier.

```cpp
#include <iostream>

void incrementer(int* ptr) {
    if (ptr != nullptr) {
        (*ptr)++;   // parentheses : on incremente la valeur pointee
    }
}

int main() {
    int compteur{10};

    incrementer(&compteur);
    incrementer(nullptr);       // sans effet, grace au test
    std::cout << compteur << '\n';

    return 0;
}
```

Résultat :

```
11
```

- À l'appel, on transmet l'**adresse** de la variable : `incrementer(&compteur)`.
- Les parenthèses de `(*ptr)++` sont nécessaires : sans elles, `*ptr++` ferait avancer le **pointeur** au lieu d'incrémenter la valeur pointée.

> **Conseil :** pour modifier une variable, la **référence** est plus simple (chapitre 5). Le pointeur se justifie quand l'**absence d'objet** (`nullptr`) est un cas possible.

3. Pointeurs et tableaux

Dans la plupart des expressions, le nom d'un tableau est converti en pointeur vers son premier élément. `ptr + 1` désigne l'élément **suivant**, et non l'octet suivant.

```cpp
#include <iostream>

int main() {
    int valeurs[3]{10, 20, 30};

    int* ptr{valeurs};

    std::cout << *ptr << '\n';
    std::cout << *(ptr + 1) << '\n';
    std::cout << *(ptr + 2) << '\n';

    return 0;
}
```

Résultat :

```
10
20
30
```

Dans un programme courant, on préfère les indices, la boucle sur un intervalle et `std::vector`.

4. Pointeur vers une structure : `->`

Pour un pointeur vers une `struct` ou un objet, `ptr->membre` est un raccourci pour `(*ptr).membre`. Cette notation sert dans le cours de POO.

```cpp
#include <iostream>

struct Mesure {
    int capteur{};
    double valeur{};
};

int main() {
    Mesure m{3, 21.5};
    Mesure* ptr{&m};

    std::cout << ptr->capteur << " : " << ptr->valeur << '\n';

    return 0;
}
```

Résultat :

```
3 : 21.5
```

5. Valeur, référence ou pointeur ?

5.1 Comparaison des notions

Notion	Déclaration	Peut être nul ?	Peut changer de cible ?	Accès
Variable	`int x{10};`	—	—	`x`
Référence	`int& ref{x};`	Non	Non	`ref`
Pointeur	`int* ptr{&x};`	Oui (`nullptr`)	Oui	`*ptr` , `ptr->membre`

5.2 Trois façons de transmettre un paramètre

La même fonction « doubler » peut s'écrire de trois façons :

```cpp
#include <iostream>

void doublerCopie(int n) {          // par valeur : travaille sur une copie
    n *= 2;
}

void doublerReference(int& n) {     // par reference : agit sur l'original
    n *= 2;
}

void doublerPointeur(int* p) {      // par pointeur : agit sur l'original, si p n'est pas nul
    if (p != nullptr) {
        *p *= 2;
    }
}

int main() {
    int a{5};
    int b{5};
    int c{5};

    doublerCopie(a);
    doublerReference(b);
    doublerPointeur(&c);

    std::cout << a << ' ' << b << ' ' << c << '\n';

    return 0;
}
```

Résultat :

```
5 10 10
```

Passage	Appel	Dans la fonction	L'original est-il modifié ?	« Pas d'objet » possible ?
Par valeur	`doublerCopie(a)`	`n`	Non	Non
Par référence	`doublerReference(b)`	`n`	Oui	Non
Par référence constante	`f(objet)`	`objet` (lecture seule)	Non (interdit)	Non
Par pointeur	`doublerPointeur(&c)`	`*p`	Oui	Oui (`nullptr`)

Méthode : pour lire une petite valeur, passage par valeur ; pour lire un gros objet, référence constante ; pour modifier, référence ; pour un objet qui peut être **absent**, pointeur, testé contre `nullptr`.

Erreurs fréquentes

Erreur	Effet	Correction
Pointeur non initialisé (`int* p;`)	Désigne n'importe où : comportement indéfini	`int* p{nullptr};` ou `int* p{&x};`
Déréférencer `nullptr`	Plantage	Tester `p != nullptr` avant
`*ptr++` pour incrémenter la valeur	Le pointeur avance, la valeur ne change pas	`(*ptr)++`
`int* a, b;`	Seul `a` est un pointeur ; `b` est un `int`	Une déclaration par ligne
Oublier `&` à l'appel : `doublerPointeur(c)`	Erreur de compilation (un `int` n'est pas une adresse)	`doublerPointeur(&c)`
Garder l'adresse d'une variable locale après la fin de son bloc	La variable n'existe plus : comportement indéfini	Ne pas utiliser un pointeur au-delà de la portée de la variable

Synthèse

- `&x` : adresse de `x` ; `int* p{&x}` mémorise cette adresse ; `*p` accède à `x` ; `p->membre` = `(*p).membre`.
- Un pointeur peut être nul (`nullptr`) : on le teste avant de le déréférencer ; on l'initialise toujours.
- `&` et `*` changent de sens selon le contexte : déclaration, expression, ou opération bit à bit.
- Paramètres : valeur (copie), référence (modifie l'original), référence constante (lecture sans copie), pointeur (modifie l'original, objet éventuellement absent).
- Pour modifier un paramètre, la référence est plus simple ; le pointeur sert quand l'absence d'objet est possible.